

ForgeOS

A manufacturing operations and accountability system connecting planned work to visible factory execution.

ForgeOS provides one operational view of work, ownership, due time, blockers, evidence, verification, and closure. The current repository is an interactive, responsive product implementation shaped by the client workbook analysis and designed as the foundation for a controlled production rollout.

Prepared	19 July 2026
Repository version	0.1.0
Current status	Interactive implementation using typed local data and browser persistence
Production status	Identity, database, file storage, notifications, and enterprise integrations remain to be implemented

Executive summary

ForgeOS is positioned between planning systems and factory execution. It turns operational activity into accountable work with one owner, one due time, a visible state, an escalation path, and a verification step before closure.

The current build demonstrates the complete product direction across command, work execution, projects, assurance, workforce, KPI, document, supplier, and administrative views. It is role aware, responsive across desktop, tablet, and mobile, and includes local work creation and state transition flows.

This build should be treated as a validated product foundation rather than a production system of record. Its next stage is to replace local seed data and browser storage with client-approved identity, database, file, notification, audit, and integration services.

Business focus	Factory execution, exception visibility, ownership, escalation, evidence, and verified closure.
Current delivery	A working Next.js application with 11 product modules, five role views, responsive navigation, search, dialogs, drawers, and local persistence.
Recommended next step	Approve a controlled pilot scope and define the authoritative data, identity, workflow, evidence, and integration contracts.

Operational problem

Fragmented operational truth

Department, project, KPI, quality, machine, workforce, safety, meeting, and communication records are spread across multiple workbook structures. The information is valuable, but it does not by itself create a live execution system.

Reporting after the event

When progress is assembled separately for reviews, leaders receive status but not always the underlying owner, blocker, evidence, or next action needed to change the outcome.

Unclear closure

Work can appear complete without a consistent verification gate, supporting evidence, or a durable record of who accepted and checked the result.

Disconnected systems

ERP, HRMS, quality, machine, document, and communication systems may remain authoritative in their own domains, while the factory still needs a shared layer for accountable execution.

Proposed solution

ForgeOS establishes a common work object across departments. Every meaningful action can carry an owner, supervisor, location, unit, shift, due time, priority, state, blocker, checklist, evidence count, project link, asset link, and last update.

- **Exceptions first:** blocked, overdue, unsafe, nonconforming, and at-risk work is surfaced before healthy work.
- **Accountability visible:** owner, supervisor, due time, and current state remain present at the point of action.
- **Role-aware views:** each role sees the modules required for its operational responsibility.
- **Verification before closure:** work moves through execution and an independent verification state before it is closed.
- **Integration rather than duplication:** production implementation should connect to authoritative client systems through governed contracts.

Completed modules in the current build

The modules below are implemented as interactive application views. Command and Work include the deepest operational interaction. The remaining modules use typed, realistic seed records, summaries, filters, record inspection, and linked action entry points.

Current product capability by module

Module	Implemented capability	Current boundary
Command Center	Scoped unit and shift metrics, exception-led work queue, project exposure, workforce coverage, and work lifecycle context.	Calculated from typed local seed and user-created work.
Work	List and board views, filtering, work creation, ownership, due time, blocker handling, progress, detail drawer, and state transitions.	Work persists in the current browser through localStorage.
Projects	Delivery stage, completion, risk, trend, milestones, open actions, exposure ranking, and record inspection.	No live project system connection.
Quality	Major and minor nonconformities, observations, CAPA, holds, ageing, ownership, state, and linked work entry points.	Evidence and quality records are representative data.
Maintenance	Asset health, runtime, service date, calibration or breakdown status, open work, and accountable owner.	No CMMS, machine, or sensor integration.
Safety	Incidents, near misses, unsafe conditions, toolbox talks, permits, severity, due date, and ownership.	No permit or incident system integration.
Workforce	Employee hierarchy, department, unit, shift, presence, skills coverage, reporting line, and assigned work.	No HRMS or biometric connection.
KPIs	Department measures, target, actual, frequency, owner, state, trend, and recovery action count.	No automated source-system actuals.
Documents	Controlled document code, revision, owner, area, effective date, status, and acknowledgement state.	No document storage, approval engine, or file delivery.

Module	Implemented capability	Current boundary
Suppliers	Supplier category, delivery score, quality score, open issues, next commitment, ownership, and risk.	No supplier portal or ERP procurement feed.
Settings	Product surfaces for roles, workflow rules, integration readiness, and governance records.	Configuration is demonstrative, not production persisted.
Shared platform	Role-based navigation, unit and shift context, global search, notifications, offline status, responsive navigation, native dialogs, drawers, and accessible shared controls.	No production authentication, authorization service, or notification delivery service.

Accountability workflow

ForgeOS uses one operational lifecycle so every role understands what has happened, what is required next, and who owns the decision.

01	Assigned	Work is created with one accountable owner, supervisor, due time, location, unit, shift, priority, and relevant project or asset context.
02	Accepted	The owner accepts responsibility and begins execution. The current implementation represents this as a transition from Not started to In progress.
03	Executed	The owner records progress, checklist completion, notes, blockers, and evidence references while work remains active.
04	Submitted	Completed execution moves to Awaiting verification. It remains open and visible until a permitted reviewer checks the result.
05	Verified	A supervisor, department head, executive, or administrator can return the work for correction or verify and close it, subject to role access.
06	Closed	The finished record is presented as read only. Production implementation should preserve an immutable event history, evidence files, actor identity, and timestamps.

ACCURACY NOTE

State transitions and same-browser persistence are functional in the current repository. Evidence counts, attachment controls, and lifecycle history are demonstrated in the interface, but production file upload, durable audit storage, approval signatures, and cross-device synchronization are not yet implemented.

User roles

Role views implemented in the application

Role	Primary responsibility	Current module access
Operator	Execute assigned work, report blockers, submit evidence, and access safety and controlled documents.	Command Center, Work, Safety, Documents
Supervisor	Plan shift work, allocate ownership, remove blockers, review execution, and verify completion.	Command Center, Work, Projects, Quality, Maintenance, Safety, Workforce, Documents
HOD	Control department performance, workforce coverage, project exposure, KPI recovery, supplier issues, and escalations.	All operational modules except Settings
Executive	Review cross-functional exceptions, delivery risk, workforce coverage, KPI performance, quality, safety, and supplier exposure.	Command Center, Work, Projects, Quality, Safety, Workforce, KPIs, Suppliers
Admin	Manage product configuration, role access, workflow rules, integration readiness, and governance.	All 11 modules

Source workbooks analyzed

The product model was shaped from three client workbooks containing 40 worksheets in total. The current repository preserves selected audited totals and operating concepts as typed seed data. It does not import these files at runtime.

Workbook analysis scope

Source workbook	Worksheets	Operational content used to shape ForgeOS
Master -Department & Project Folder's.xlsx	21	Master folders, projects, plant machines, instruments, lifting tools, toolbox talks, nonconformities, complaints, cost of quality, project reviews, audits, meeting records, and customer and vendor communication matrices.
Objective & KOI - Master File R0.xlsx	13	Executive and achievement views plus objectives and KPI structures for HR, quality, business development, PMG, production, machine shop, welding, maintenance, finance, and supply chain.
Employee Data & Org Structure - R5 - UPATED 25.6.26.xlsx	6	Employee grades, employee master data, final organization structure, and operational manpower requirements.

AUDITED TOTALS REPRESENTED IN THE REPOSITORY

83 employees, a workforce requirement of 92, a gap of 9, 83 audit findings, 22 major nonconformities, 26 KPI records, and 31 employees linked across 7 project codes.

Technical architecture

Application framework	Next.js 16.2.10 with React 19.2.7 and TypeScript 5.9.3.
Delivery model	Next.js static export with support for a repository base path in GitHub Actions. The generated application can be hosted on an HTTPS static web host.
Application layer	A role-aware client application composed of Command Center, Work, secondary operational views, shared dialogs, drawers, status components, fields, navigation, and feedback patterns.
Data model	Typed TypeScript records for work, projects, quality, assets, safety, employees, KPIs, documents, suppliers, activities, and workflow state.
Current persistence	Seed records are bundled with the application. User-created and updated work records persist in localStorage on the same browser and device.
Interaction model	Native dialog semantics, focus management, keyboard navigation, responsive desktop, tablet, and mobile layouts, reduced motion support, and forced color support.
Production services required	Identity provider, server-side authorization, relational or document database, object storage, notification service, immutable audit events, background synchronization, monitoring, backups, and enterprise system connectors.

Testing and quality

The repository contains unit, component, integration, responsive browser, and accessibility test infrastructure. Verification performed for this handoff is listed below.

Verification status on 19 July 2026

Check	Scope	Result
TypeScript	Strict project typecheck using TypeScript with no emitted output.	Passed
Vitest	Metrics, seed integrity, application navigation and search, Work filtering, board behavior, and record opening.	4 files, 10 tests passed
Browser harness	Playwright Core script for desktop, tablet, mobile, overflow, work creation, state transitions, role access, global search, offline status, runtime errors, semantic state treatments, and axe-core WCAG checks.	Included in repository and intended as a release gate

Before a production pilot, CI should require typecheck, unit and component tests, static build, browser workflow tests, accessibility scanning, security scanning, and integration contract tests against non-production client systems.

Deployment

Current application

- Install with **pnpm install**.
- Run locally with **pnpm dev**.
- Verify with **pnpm typecheck**, **pnpm test**, and the browser harness while the application is served on port 3000.
- Build as a static export with **pnpm build**.
- Host the generated static output on an approved HTTPS web host.

Production platform

- Place authentication and authorization ahead of every protected route and service call.
- Store operational records and audit events in managed, backed-up production data services.
- Store evidence in governed object storage with access control, retention, and malware scanning.
- Introduce observable APIs and connectors for ERP, MES, HRMS, QMS, biometric, document, and machine systems as approved.
- Provide monitored offline synchronization rather than browser-only local persistence.

Implementation roadmap

The roadmap is intentionally sequenced by operational risk and data authority. Dates and commercial estimates should be agreed only after client system discovery and pilot scope confirmation.

01	Align	Confirm sponsor, pilot department, unit, shift, success measures, workflow owners, verification authority, data owners, and production deployment constraints.
02	Contract	Define the work, project, employee, asset, quality, KPI, document, supplier, evidence, and audit schemas. Identify each authoritative source and reconciliation rule.
03	Foundation	Implement identity, role-based access control, APIs, production database, object storage, immutable audit events, notifications, monitoring, backup, and environment management.
04	Pilot	Deploy Work, Command Center, Quality, and Workforce for one controlled operational scope. Train users, run parallel validation, measure adoption, and resolve workflow exceptions.
05	Integrate	Connect approved ERP, MES, HRMS, QMS, document, biometric, supplier, and machine sources in priority order. Add governed offline synchronization where shop floor conditions require it.
06	Scale	Expand department and project coverage, formalize support and change control, monitor data quality, and use verified operational history for performance improvement.

Client value

ForgeOS is designed to create value by connecting work and evidence, not by adding another reporting layer. Benefits depend on adoption, data quality, workflow governance, and successful system integration.

One accountability language

Every department works with the same owner, due time, state, blocker, evidence, and verification concepts.

Less status assembly

Dashboards can be derived from live work records instead of recreated separately after execution.

Cross-functional visibility

Projects, quality, maintenance, safety, workforce, KPIs, documents, and suppliers can connect through shared work.

Faster exception response

Blocked, overdue, unsafe, nonconforming, and at-risk work can lead the operational view.

Verifiable closure

Execution and review are separated so critical work is checked before it is treated as complete.

Integration foundation

The product model provides a practical execution layer while preserving authoritative client systems.

Inputs needed to begin a controlled pilot

- Executive sponsor and accountable client product owner
 - Named pilot department, unit, shift, and user group
 - Approved workflow states, closure rules, and verification authority
 - Identity provider and role mapping
 - Authoritative master data and record ownership
 - ERP, MES, HRMS, QMS, document, and machine system owners
 - Evidence retention, security, privacy, and audit requirements
 - Hosting, network, device, browser, and offline constraints
 - Pilot success measures and acceptance criteria
 - Support, training, rollout, and change-control ownership
-

ForgeOS client overview

Prepared from the repository state and workbook analysis available on 19 July 2026.

This document distinguishes the completed interactive implementation from production services and integrations that remain to be delivered.